

One-to-Many Relationships in Entity Framework Core

In this article, I will discuss How to Configure **One-to-Many Relationships between two Entities in Entity Framework Core** with Examples. Please read our previous article discussing [How to Configure One-to-One Relationships between two entities in EF Core](#) with Examples.

What is the One-to-Many Relationship in Entity Framework Core?

A One-to-Many relationship in Entity Framework Core represents a scenario where one entity (the principal) is associated with zero or many instances of another entity (the dependent). This type of relationship is useful when one record in a table is linked to multiple records in another table.

For example, consider an application that manages Orders and their corresponding Order Items. Each Order can have multiple Order Items, but each Order Item belongs to only one Order. This is a classic One-to-Many relationship example in our real-time e-commerce application.

Consider another real-time example: an application that manages departments and employees within a company. Each Department can have multiple Employees, but each Employee belongs to only one Department. This is another example of a One-to-Many relationship.

Guidelines to Implement One-to-Many Relationships in Entity Framework Core

The following are the key guidelines for implementing One-to-Many relationships in Entity Framework Core:

- **Principal and Dependent Entities:** In a One-to-Many relationship, the principal entity is the “one” side, and the dependent entity is the “many” side, i.e., One Principal Entity can have multiple Dependent Entities. So, first, you need to determine which entity is the principal (e.g., Department, Order) and which is the dependent entity (e.g., Employee, OrderItem).
- **Foreign Key Constraints:** The dependent entity contains the foreign key referencing the principal entity.
- **Optional vs. Required Relationship:** Define whether the relationship is optional or required. The dependent entity can exist without the principal entity if it's optional. For example, an Employee might be required to belong to a Department, or it might be optional.
- **Navigation Properties:** The Principal Entity will contain a collection navigation property pointing to the Dependent entity. The dependent entity will have one reference navigation property pointing to the Principal entity.
- **Database Schema:** In the database, One-to-Many relationships are represented by placing a foreign key constraint in the dependent table pointing to the principal table.

Real-Time Example to Understand One-to-Many Relationships in EF Core

Let's consider an example of Order and OrderItem entities to demonstrate how to implement One-to-Many relationships in Entity Framework Core. I will show you three approaches to implementing One-to-Many Relationships in EF Core. They are as follows:

1. **Implementing One-to-Many Relationships without Data Annotations or Fluent API**
2. **Implementing One-to-Many Relationships with Data Annotations**
3. **Implementing One-to-Many Relationships with Fluent API**

Implementing One-to-Many Relationships Without Data Annotations or Fluent API

EF Core can sometimes manage relationships automatically without any explicit configuration. However, if we don't use data annotations or Fluent API, EF Core will attempt to create the default relationships based on naming conventions. Let's understand how to implement the One-to-Many relationship with default EF Core conventions.

Creating Entities:

We want to establish a One-to-Many relationship between Order and OrderItem entities.

Order Entity

Create a class file named **Order.cs** within the Entities folder with the following code. This will be our principal entity.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Order
    {
        public int Id { get; set; }
        public DateTime OrderDate { get; set; }
        public string OrderNumber { get; set; }
        public List<OrderItem> OrderItems { get; set; } // Navigation
    }
}
```

OrderItem Entity

Next, create a class file named OrderItem.cs within the Entities folder. This will be our dependent entity.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderItem
    {
        public int Id { get; set; }
        public string ProductName { get; set; }
        public int OrderId { get; set; } // FK, Required Relationship
        public Order Order { get; set; } // Navigation property
    }
}
```

With Default Convention:

- EF Core can detect the One-to-Many relationship based on the presence of navigation properties and foreign key properties.
- The OrderItem entity has a foreign key OrderId and a navigation property Order.
- The Order entity has a navigation property, OrderItems, to represent the multiple items associated with an order.

DbContext Class:

Next, modify the EFCoreDbContext class as follows. Here, we add both Order and OrderItem as DbSet properties so that EF Core can generate the required database tables with a One-to-Many relationship.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
```

```

{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=OrderDB;Trusted_Connection=True;TrustServerCertificate=True;")
        }
        public DbSet<Order> Orders { get; set; }
        public DbSet<OrderItem> OrderItems { get; set; }
    }
}

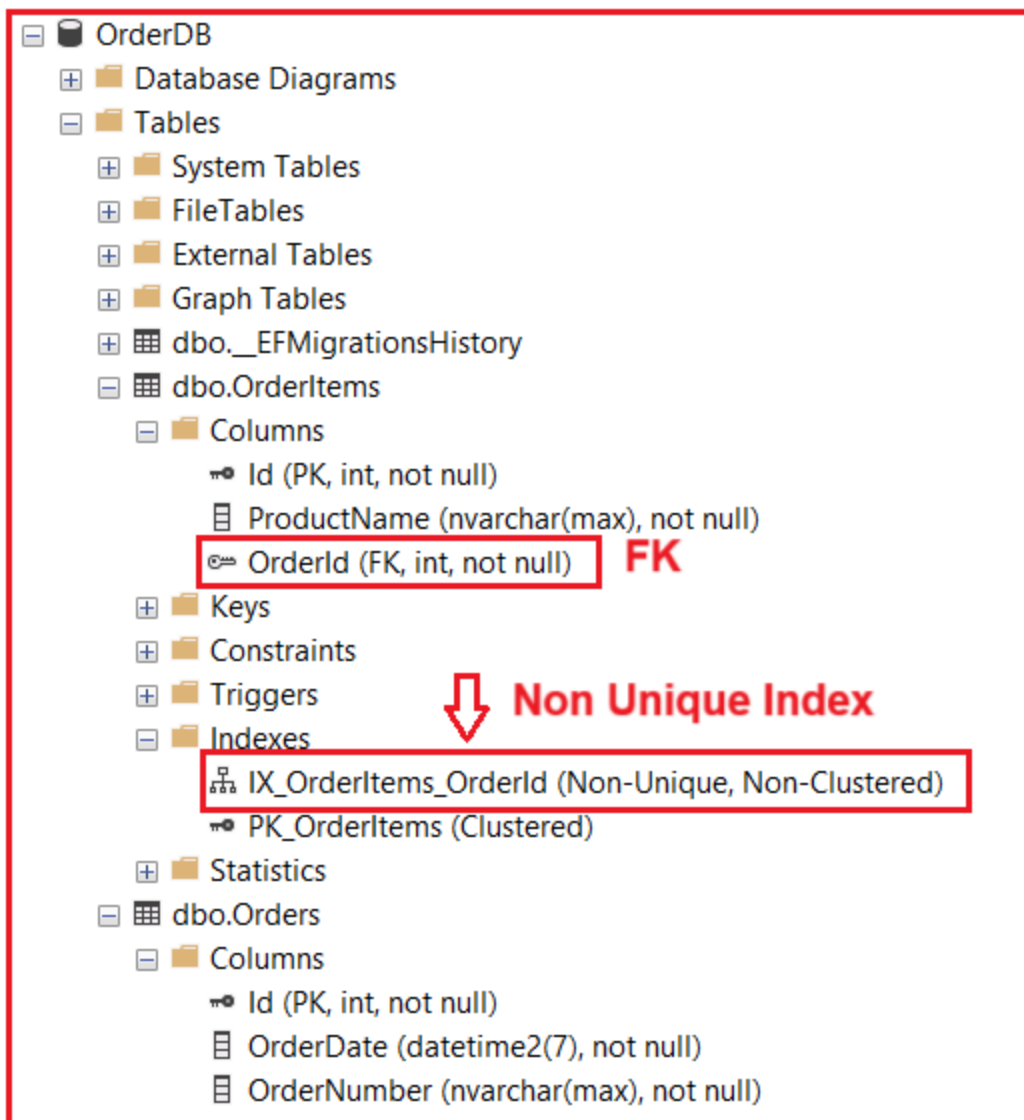
```

Generating the Migration:

After the changes, run the following commands in the Package Manager Console:

- **Add-Migration CreateOrderTables**
- **Update-Database**

Once you execute the above commands, the database should be created with the required Orders and OrderItems tables with a One-to-Many relationship. In the OrderItems table, the OrderId column is created as a foreign key pointing to the Orders table's primary key column, as shown in the image below.



As you can see, it creates a Non-Unique and Non-Clustered Index in the Foreign Key OrderId column. That means this column can hold duplicate values, i.e., we can have the same OrderId for multiple OrderItems, i.e., One OrderId can have multiple OrderItems. This is how it implements One-to-Many Relationships between Order and OrderItem entities.

Limitations:

- Without explicit configuration, EF Core relies on conventions that might not always align with your intended schema.
- It's recommended to use Data Annotations or Fluent API for clarity and to ensure the relationship is correctly established.

Implementing One-to-Many Relationships Using Data Annotation Attributes in EF Core

We can use Data Annotation Attributes to define One-to-Many relationships explicitly between two entities in Entity Framework Core.

Entities with Data Annotations:

First, modify the **Order.cs** file as follows:

```

using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Order
    {
        [Key]
        public int Id { get; set; } // Primary Key
        public DateTime OrderDate { get; set; }
        public string OrderNumber { get; set; }
        public List<OrderItem> OrderItems { get; set; } // Navigation
    }
}

```

Next, modify the **OrderItem.cs** file as follows:

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderItem
    {
        [Key]
        public int Id { get; set; }
        public string ProductName { get; set; }
        [ForeignKey("OrderId")]
        public int OrderId { get; set; } // FK, Required Relationship
        public Order Order { get; set; } // Navigation property
    }
}

```

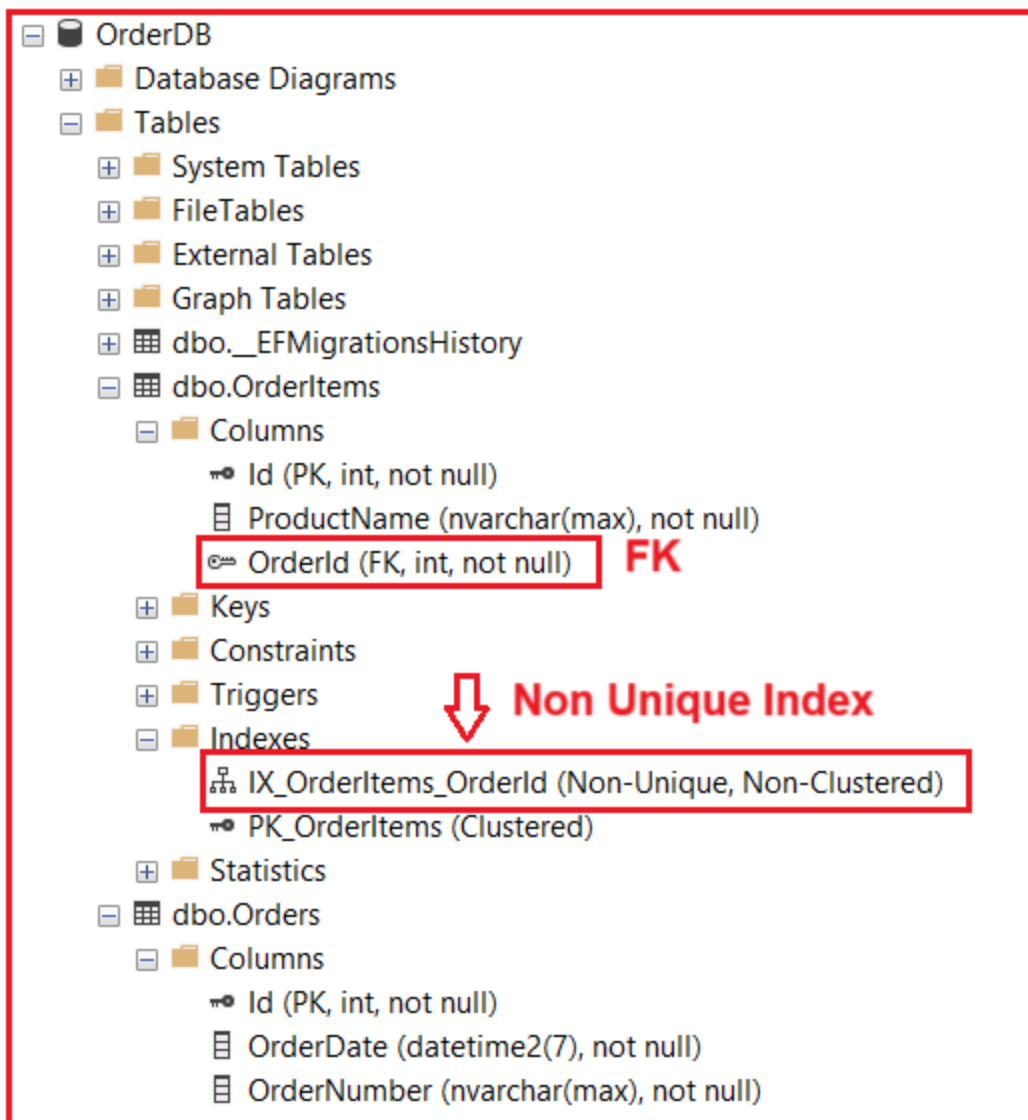
Explanation:

- The [Key] attribute marks the primary key for each entity.
- The [ForeignKey("OrderId")] attribute in the OrderItem entity explicitly declares OrderId as a foreign key.

This configuration ensures that each OrderItem is linked to exactly one Order, and one Order can have many OrderItems, enforcing a One-to-Many relationship between them.

Generating Migration:

With the above changes, open the Package Manager Console and execute the Add-Migration and Update-Database commands. Once the commands are executed, please verify the database. The Orders and OrderItems tables with the correct One-to-Many relationship, as shown in the image below.



Note: The [ForeignKey] attribute can be applied either to the foreign key property (OrderId) or to the navigation property (Order). Applying it to the foreign key property is more explicit.

Implementing One-to-Many Relationships in EF Core Using Fluent API

The Fluent API provides a more expressive way to configure the model within the OnModelCreating method of the DbContext class. This is the recommended approach for complex configurations. First, modify the entities by removing the [Key] and [ForeignKey] attributes from the model properties:

Order.cs

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Order
    {
        public int Id { get; set; } // Primary Key
        public DateTime OrderDate { get; set; }
        public string OrderNumber { get; set; }
        public List<OrderItem> OrderItems { get; set; } // Navigation
    }
}
```

```
    }
}
```

OrderItem.cs

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderItem
    {
        public int Id { get; set; }
        public string ProductName { get; set; }
        public int OrderId { get; set; } // FK, Required Relationship
        public Order Order { get; set; } // Navigation property
    }
}
```

DbContext Configuration:

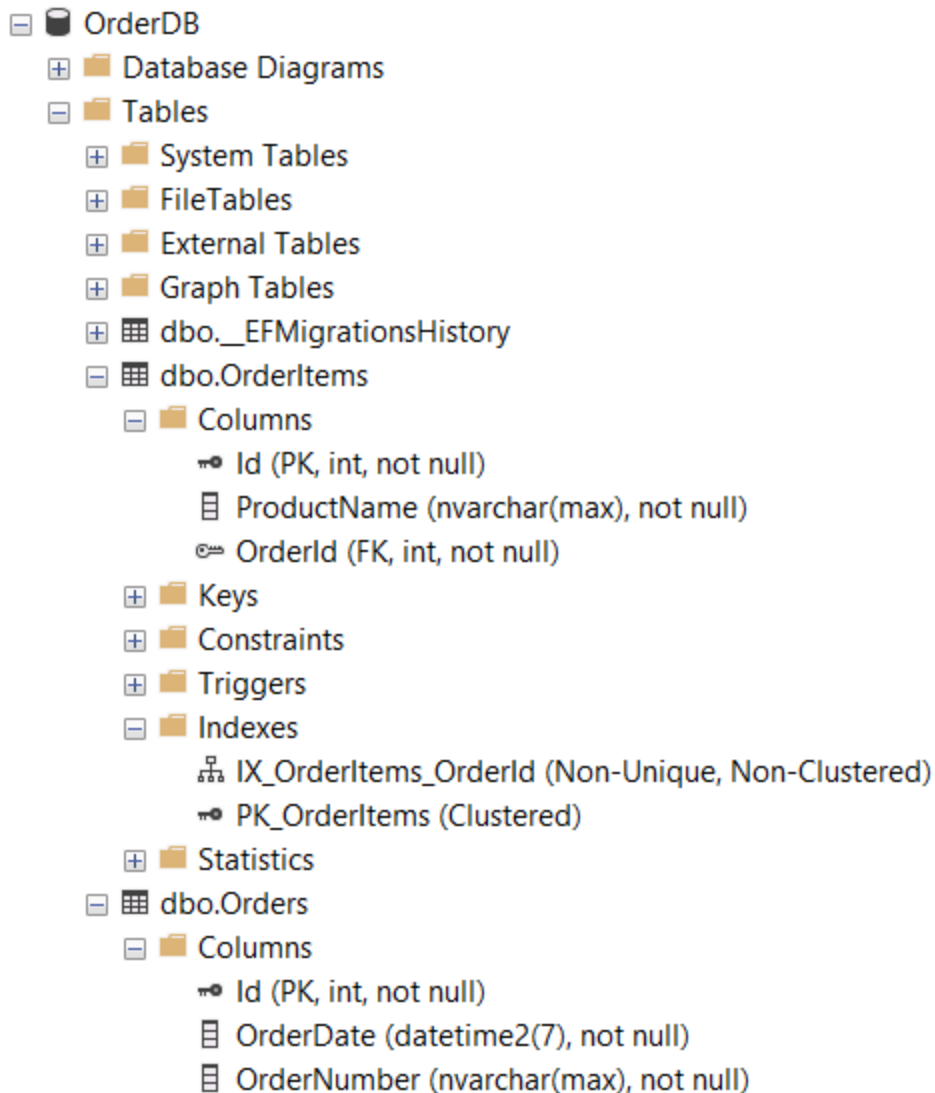
We need to configure the One-to-Many relationship between Order and OrderItem using Fluent API by overriding the OnModelCreating method of the DbContext class. So, please modify the EFCoreDbContext class as follows:

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=OrderDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Order>() //Refers to the Order entity.
                .HasMany(o => o.OrderItems) // Order has many OrderItems
                .WithOne(oi => oi.Order) // Each OrderItem has one Order
                .HasForeignKey(oi => oi.OrderId); // OrderId is the FK in OrderItem
        }
        public DbSet<Order> Orders { get; set; }
        public DbSet<OrderItem> OrderItems { get; set; }
    }
}
```

This configuration clearly defines how an Order can have multiple Order Items, each associated with one Order.

Migration and Database Update:

After setting up the entities and DbContext, open the Package Manager Console and execute the Add-Migration and Update-Database commands. Once the commands are executed, verify the database to ensure the schema reflects the One-to-Many relationship between Order and OrderItem.



How Do We Configure Cascade Delete using EF Core for One-to-Many Relationships?

In Entity Framework Core (EF Core), the `OnDelete` method configures the delete behavior for related entities when the principal entity is deleted. This behavior is commonly referred to as cascading actions. When defining relationships between entities, particularly foreign key relationships, it's important to determine what should happen when a referenced entity (the principal) is deleted. The `OnDelete` method allows you to specify this behavior.

Modify the `EFCoreDbContext`:

For example, if an `Order` is deleted from the `Orders` table, then automatically, we need to delete all the related `OrderItems` belonging to that `Order` from the `OrderItems` table. To implement this, we need to use the `OnDelete` Fluent API method and set the `DeleteBehavior` to `Cascade`. So, please modify the `EFCoreDbContext` class as follows. Here, you can see we are using the `OnDelete` method; to this method, we are passing the `DeleteBehavior.Cascade` enum named constant.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
```

```

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    // Configuring the Connection String
    optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=OrderDB;Trusted_Connection=True;TrustServerCertificate=False;Encrypt=True;
}
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Fluent API Configuration
    // Configure One to Many Relationships Between Order and OrderItem
    modelBuilder.Entity<Order>()
        .HasMany(o => o.OrderItems) // Order has many OrderItems, specific
of the relationship
        .WithOne(oi => oi.Order)    // OrderItem is associated with one Or
'one' side of the relationship
        .HasForeignKey(oi => oi.OrderId) // OrderId is the Foreign key in
specifies the foreign key
        .OnDelete(DeleteBehavior.Cascade); // This will delete the child r
parent record is deleted
    }
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }
}
}

```

Understanding the DeleteBehavior

The OnDelete method uses the DeleteBehavior parameter, which determines how a delete operation affects dependent entities when the principal is deleted. If you go to the definition of DeleteBehavior, you will see it is an enum with the following values.

```
public enum DeleteBehavior
{
    ... ClientSetNull,
    ... Restrict,
    ... SetNull,
    ... Cascade,
    ... ClientCascade,
    ... NoAction,
    ... ClientNoAction
}
```

The DeleteBehavior enum has the following values:

Cascade:

- When the principal entity (Order) is deleted, all related dependent entities (e.g., related OrderItems) will also be deleted.
- This behavior is enforced at both the database and EF Core levels. It ensures that child records cannot exist without the parent record.
- The database applies a cascading delete constraint, so when a parent is deleted, all dependents are also removed automatically.

ClientSetNull:

- When the principal entity is deleted, the foreign key value in the dependent entities (e.g., OrderId in OrderItem) is set to null, but this change happens only in EF Core's in-memory context before saving to the database.
- If the foreign key property in the dependent entity is non-nullable, an exception will be thrown during SaveChanges() because EF Core cannot set the foreign key to null.

Restrict:

- This prevents the principal entity from being deleted if any related dependent entities exist.
- It helps prevent accidental data loss by ensuring that parent entities cannot be deleted when dependents are present.
- This behavior is enforced both in EF Core and the database. If you try to delete a parent entity with existing dependents, EF Core will throw an exception.

SetNull:

- When the principal entity is deleted, the foreign key value in the dependent entities (e.g., OrderId in OrderItem) is set to null.
- This behavior is enforced both at the database level and in EF Core. When deleting the parent, the foreign key columns in the dependent entities are updated to null, and this occurs directly in the database.
- Unlike ClientSetNull, which only affects in-memory entities, SetNull operates on persisted data in the database.

ClientCascade:

- This is similar to Cascade, but the cascading delete happens only on the client side (EF Core context), meaning that the dependent entities are deleted in memory when the principal entity is deleted.
- However, cascading deletes do not happen at the database level, meaning that if you delete the principal entity from EF Core, the dependents are removed in memory, but the database will not automatically delete the dependents.

NoAction:

- When the principal entity is deleted, no action is taken on the dependent entities, meaning that the foreign key values in the dependents remain unchanged.
- If the foreign key is non-nullable and you attempt to delete the principal entity without handling the dependents, the database may violate the foreign key constraint.
- This behavior indicates that no cascade or update action is applied, and it is up to you to handle the situation appropriately to avoid errors.

ClientNoAction:

- Similar to NoAction, but this behavior only applies to entities being tracked by the EF Core context.
- The database does not apply any cascade or update action, and no changes are made to the foreign key relationships at the database level. Instead, this behavior is enforced only within EF Core's in-memory context.
- You must handle dependent entities manually, as neither EF Core nor the database takes any action to change foreign key values.

Migration and Database Update

With the above changes, open the Package Manager Console and execute the Add-Migration and Update-Database commands.

Adding Orders into the Database

To understand the Cascade Delete behavior, we need to add a few orders and related order items to the database. So, please modify the Program class as follows to add two orders, each having two order items, into the database. The following example code is self-explained. Please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
```

```

// Step 1: Initialize the DbContext
using var context = new EFCoreDbContext();
// Step 2: Create two orders with two order items each
// First Order
var order1 = new Order
{
    //Order Id will be 1
    OrderNumber = "ORD001", // Unique order number
    OrderDate = DateTime.Now,
    OrderItems = new List<OrderItem>
    {
        new OrderItem { ProductName = "Product A" }, // First
item in the order
        new OrderItem { ProductName = "Product B" } //
Second item in the order
    }
};
// Second Order
var order2 = new Order
{
    //Order Id will be 2
    OrderNumber = "ORD002", // Unique order number
    OrderDate = DateTime.Now,
    OrderItems = new List<OrderItem>
    {
        new OrderItem { ProductName = "Product C" }, // First
item in the order
        new OrderItem { ProductName = "Product D" } //
Second item in the order
    }
};
// Step 3: Add the orders (and their items) to the context
context.Orders.Add(order1); // Adding first order
context.Orders.Add(order2); // Adding second order
// Step 4: Save changes to the database
context.SaveChanges(); // This will save both orders and
their respective order items
// Step 5: Output to confirm the operation
Console.WriteLine("Two Orders have been created
successfully!");
Console.WriteLine($"Order 1: {order1.OrderNumber} with
{order1.OrderItems.Count} items.");
Console.WriteLine($"Order 2: {order2.OrderNumber} with
{order2.OrderItems.Count} items.");
// Step 6: Retrieve and display the created orders with their
items from the database
var orders = context.Orders
.Include(ord => ord.OrderItems) //Eager Loading the
Related Order Items
.ToList(); // Fetch all orders from the database

```

```

        Console.WriteLine("\n Fetching All Orders and Related Order
Items from DB");
        foreach (var order in orders)
        {
            Console.WriteLine($"Order ID: {order.Id}, Order Number:
{order.OrderNumber}, Order Date: {order.OrderDate}");
            // Retrieve the related order items
            foreach (var item in order.OrderItems)
            {
                Console.WriteLine($"OrderItem ID: {item.Id},
Product Name: {item.ProductName}");
            }
        }
    }
    catch (Exception ex)
    {
        // Handle any errors that occur during the operation
        Console.WriteLine($"An error occurred: {ex.Message}");
    }
    // Pause the console window to see the output
    Console.ReadKey();
}
}
}

```

Output:

```

Two Orders have been created successfully!
Order 1: ORD001 with 2 items.
Order 2: ORD002 with 2 items.

  Fetching All Orders and Related Order Items from DB

Order ID: 1, Order Number: ORD001, Order Date: 15-09-2024 19:50:48
    OrderItem ID: 1, Product Name: Product A
    OrderItem ID: 2, Product Name: Product B

Order ID: 2, Order Number: ORD002, Order Date: 15-09-2024 19:50:48
    OrderItem ID: 3, Product Name: Product C
    OrderItem ID: 4, Product Name: Product D

```

Example of Cascade Delete in Action

Now, please modify the Program class as follows to demonstrate how to delete an Order and ensure that the associated OrderItems are also deleted when using cascade delete.

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo

```

```

{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Step 1: Initialize the DbContext
                using var context = new EFCoreDbContext();
                // Step 2: Retrieve an order to delete (in this case, we're
                deleting the order with ID 1)
                var orderToDelete = context.Orders
                    .Include(ord => ord.OrderItems) //Eager Load the Related
Order Items
                    .FirstOrDefault(o => o.Id == 1); // Retrieve order with
ID = 1

                if (orderToDelete != null)
                {
                    // Output the order details before deletion
                    Console.WriteLine($"Order to be deleted:
{orderToDelete.OrderNumber}");
                    // Output the related order items before deletion
                    foreach (var item in orderToDelete.OrderItems)
                    {
                        Console.WriteLine($"\\tOrderItemId: {item.Id}, Product
Name: {item.ProductName}");
                    }
                    // Step 3: Delete the order
                    context.Orders.Remove(orderToDelete); // Remove the order
(this will trigger cascade delete for related order items)
                    // Step 4: Save the changes to the database
                    context.SaveChanges(); // This will delete the order and
its related order items (due to cascade delete)
                    // Output the success message
                    Console.WriteLine($"Order '{orderToDelete.OrderNumber}'
and its related items were successfully deleted.");
                }
                else
                {
                    // Output if the order with the specified ID was not
found
                    Console.WriteLine("Order not found. No deletion
performed.");
                }
                // Step 5: Verify that the order and related order items are
deleted by attempting to retrieve them again
                var deletedOrder = context.Orders.FirstOrDefault(o => o.Id ==
1); // Trying to find the deleted order
                if (deletedOrder == null)
                {

```

```

        Console.WriteLine("Order with ID 1 has been deleted from
the database.");
    }
    // Verify the related OrderItems are also deleted
    var deletedOrderItems = context.OrderItems.Where(oi =>
oi.OrderId == 1).ToList();
    if (deletedOrderItems.Count == 0)
    {
        Console.WriteLine("All related OrderItems for Order ID 1
have been deleted.");
    }
    else
    {
        Console.WriteLine("Some OrderItems for Order ID 1 are
still present, which should not happen with Cascade delete.");
    }
}
catch (Exception ex)
{
    // Handle any errors that occur during the operation
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

If we delete an Order with Order Id = 1, the associated OrderItems for that order should also be deleted automatically due to the cascade delete behavior configured via Fluent API. Now, run the application, and you should get the following output:

```

Order to be deleted: ORD001
    OrderItemId: 1, Product Name: Product A
    OrderItemId: 2, Product Name: Product B
Order 'ORD001' and its related items were successfully deleted.
Order with ID 1 has been deleted from the database.
All related OrderItems for Order ID 1 have been deleted.

```

When to Use One-to-Many Relationships in Entity Framework Core

One-to-many relationships are suitable when:

- **A Single Entity is Related to Multiple Instances of Another Entity:** For example, an order containing multiple order items, where one Order entity is associated with multiple OrderItem entities. This is common in scenarios where a parent entity (the “one” side) has a collection of related child entities (the “many” side).
- **Logical Grouping of Data:** If a single entity (like an Order) needs to group multiple related records (like OrderItems), a One-to-Many relationship is an effective way to organize the data. This ensures that the relationship between the entities is clearly defined and easy to query.
- **Parent-Child Relationships:** One-to-many is ideal for modeling hierarchical relationships, such as a Category having multiple Products or a Blog having multiple Posts. This is useful when you need to represent relationships where each entity on the “many” side is dependent on a single parent entity.

- **Data Aggregation:** In cases where you need to aggregate data, like calculating the total amount of an Order by summing up the OrderItems, One-to-Many relationships are necessary. This allows for efficient queries to compute totals or other aggregate data based on the related entities.

Choosing a One-to-Many relationship is essential when modeling real-world relationships where a single entity is expected to be connected to multiple entities. This type of relationship is one of the most commonly used in relational databases.

In the next article, I will discuss [how to Configure Many-to-Many Relationships in Entity Framework Core](#) with Examples. In this article, I explain **how to Configure One-to-Many Relationships in Entity Framework Core** with Examples. I hope you enjoyed this article on Configuring One-to-Many Relationships in EF Core.